

RythmoAI

Plateforme SaaS de Génération Automatique de Bande Rythmo Professionnelle pour le Doublage

CAHIER DES CHARGES FONCTIONNEL, TECHNIQUE ET INDUSTRIEL

Champ	Valeur
Nom du projet	RythmoAI (nom de travail)
Type de document	Cahier des charges complet — v1.0
Domaine	Post-production audiovisuelle <i>Doublage</i> Localisation
Statut	Document de cadrage — prêt pour lancement de projet
Date	05/08/2026
Diffusion	Confidentiel — Équipe projet, direction technique, prestataires
Stack imposée	Frontend HTML5/CSS3/JS natif — Backend Python/FastAPI — Déploiement Windows natif (sans Docker), lancement via scripts .bat / .ps1

Document destiné à permettre le démarrage immédiat du développement.

Sommaire

0. Préambule et Guide de Lecture

Ce document constitue le cahier des charges de référence pour la conception, le développement et le déploiement de RythmoAI, une plateforme SaaS destinée aux studios de doublage, adaptateurs, directeurs artistiques et calligraphes rythmo. Elle automatise, à l'aide de modèles d'intelligence artificielle, la génération d'une bande rythmo professionnelle (également appelée bande LRD — Lecture Rythmo Défilante) synchronisée avec l'image, tout en laissant à l'utilisateur un contrôle éditorial complet.

Ce document s'adresse à : la direction de projet, l'équipe technique (backend, frontend, MLOps/IA, DevOps), les designers UX/UI, les experts métiers du doublage consultés, ainsi qu'aux prestataires ou partenaires impliqués dans le développement.

Niveau d'exigence

Le document est rédigé au niveau « industriel » : chaque brique technique est spécifiée avec un niveau de détail suffisant pour permettre l'écriture immédiate de tickets de développement (User Stories, schémas de base de données, contrats d'API, pipelines). Les choix technologiques imposés par le commanditaire (HTML5/CSS3/JS natif côté frontend, Python/FastAPI côté backend) sont respectés strictement ; les choix laissés libres (IA, infrastructure, CI/CD) sont argumentés selon l'état de l'art à la date de rédaction.

Conventions

- **MUST SHOULD COULD** — priorisation MoSCoW utilisée dans le backlog (Must have, Should have, Could have, Won't have for now).
- **Bande rythmo** — bande de texte défilante synchronisée à l'image, calée sur les mouvements labiaux et les codes de doublage (crochets d'entrée/sortie, italique pour off, majuscules pour les cris, etc.), utilisée par le comédien pendant l'enregistrement.
- **LRD** — Lecture Rythmo Défilante, nom historique du procédé inventé par la société Cinétype.

Contrainte de déploiement (mise à jour du cadrage)

L'application ne doit pas reposer sur Docker ni sur toute forme de conteneurisation. Le backend, les workers et les services associés doivent pouvoir s'installer et s'exécuter nativement sous Windows, avec un démarrage simplifié via des scripts .bat et .ps1 fournis avec le projet (voir chapitre 18 « Infrastructure et Déploiement Windows »). Cette contrainte remplace les mentions antérieures de Docker/Kubernetes présentes dans les versions précédentes du cadrage technique.

Structure du document — le document suit une progression classique de cadrage produit (vision, besoins, personas, benchmark), puis descend vers l'architecture technique complète (backend, frontend, IA, données, API, pipelines), la conception UX/UI, les exigences transverses (sécurité, performance), l'infrastructure et les opérations (déploiement Windows natif, CI/CD), puis la gestion de projet (roadmap, backlog, budget, planning, risques) et enfin les évolutions futures.

1. Vision Produit

1.1 Contexte et problématique

La bande rythmo est l'outil central du doublage francophone et de plusieurs industries de doublage européennes (procédé né en France dans les années 1950). Elle est aujourd'hui produite manuellement par des « calligraphes rythmo » qui, à partir d'une détection d'auteur (repérage), du script traduit/adapté et de la vidéo, tracent au pixel près chaque réplique en tenant compte des ouvertures labiales, des respirations, des silences et des codes conventionnels du métier. Ce travail est long (plusieurs heures par minute de programme), coûteux, et dépend d'une expertise rare et non standardisée d'un studio à l'autre.

Les studios de doublage subissent une pression croissante : explosion du volume de contenus à localiser (plateformes de streaming), délais de livraison réduits, budgets serrés, pénurie de calligraphes rythmo formés. Il n'existe à ce jour aucune solution SaaS complète combinant transcription automatique de haute précision, analyse prosodique, détection de synchronisation labiale et génération de bande rythmo éditable — la plupart des studios utilisent des plugins internes (Cantar, Rythmo-Editor, VoQuent, Dubbing Wave-style) peu interopérables et sans IA embarquée.

1.2 Proposition de valeur

- **Automatisation du premier jet** — génération automatique d'une bande rythmo exploitable à 70–90 % en quelques minutes au lieu de plusieurs heures de travail manuel.
- **Précision mot-à-mot** — alignement temporel au mot près grâce à des modèles de reconnaissance vocale et d'alignement forcé de dernière génération.
- **Intelligence contextuelle** — détection automatique des respirations, hésitations, émotions et intentions afin d'assister l'adaptateur dans le calibrage du texte.
- **Édition professionnelle complète** — un éditeur web dédié permettant un contrôle total (déplacement, découpe, fusion, recalage,

codes typographiques) répondant aux standards du métier.

- **Interopérabilité** — export vers les formats utilisés par l'industrie (Cavendish, .rhythmo, EBU-STL, SRT/VTT étendus, PDF calligraphié, XML propriétaire).

- **Collaboration multi-utilisateurs** — gestion de studios, de projets et de rôles (adaptateur, directeur artistique, calligraphe, chef de projet) avec historique et validation.

1.3 Objectifs mesurables (à 18 mois)

Objectif	Indicateur	Cible
Réduction du temps de calligraphie	Temps moyen / minute de programme	-70 % vs process manuel
Précision de la transcription	WER (Word Error Rate) sur corpus FR	< 8 %
Précision de synchronisation	Écart moyen bande/audio	< 80 ms
Adoption	Studios actifs (MAU studios)	50 studios en 18 mois
Satisfaction	CSAT / NPS	NPS > 40
Disponibilité plateforme	SLA uptime	≥ 99,5 %

1.4 Périmètre (in/out)

Dans le périmètre (MVP + V1)

- Import vidéo et extraction audio automatique
- Transcription automatique multilingue avec horodatage mot-à-mot
- Diarisation (détection des locuteurs)
- Détection des respirations, pauses, hésitations
- Détection d'émotions et d'intentions (assistance, non prescriptive)
- Génération automatique de la bande rythmo avec règles métier configurables
- Éditeur de bande rythmo synchronisé à la vidéo
- Export multi-formats professionnels
- Gestion des utilisateurs, studios, projets et droits d'accès

Hors périmètre (versions ultérieures ou explicitement exclues)

- Traduction automatique du script (l'application travaille sur un texte déjà écrit en langue cible, importé ou transcrit) — une assistance à la

traduction pourra être évaluée en V2

- Doublage vocal par synthèse (voix IA) — non prévu, RythmoAI reste un outil de préparation, pas de génération vocale
- Montage vidéo/audio final (mixage, post-prod son) — hors périmètre, uniquement lecture/analyse
- Application mobile native (le MVP cible desktop web ; le responsive mobile est en lecture seule)

2. Étude des Besoins

2.1 Méthodologie

L'étude des besoins s'appuie sur (a) l'analyse du processus métier actuel de fabrication d'une bande rythmo dans un studio de doublage francophone standard, (b) l'identification des points de friction et de perte de temps, (c) le recensement des exigences réglementaires et conventionnelles propres au doublage (codes typographiques, conventions syndicales), et (d) l'analyse fonctionnelle des outils existants (voir Benchmark, chapitre 4).

2.2 Processus métier actuel (AS-IS)

Étape	Description	Acteur	Durée indicative
1	Réception vidéo + script traduit/adapté	Chef de projet	—
2	Repérage (découpage en boucles/répliques)	Détecteur / Adaptateur	1 à 2 j / 20 min
3	Calligraphie de la bande rythmo	Calligraphe rythmo	3 à 6 j / 20 min
4	Relecture et corrections	Directeur artistique	0,5 j / 20 min
5	Export et livraison au studio d'enregistrement	Chef de projet	—

Ce processus manuel représente, selon les retours de professionnels du secteur, entre 4 et 8 jours de travail qualifié pour 20 minutes de programme, un goulot d'étranglement majeur alors que la demande de contenus doublés augmente fortement avec les plateformes de streaming.

2.3 Besoins fonctionnels exprimés

Besoins des studios de doublage

- Réduire drastiquement le délai de mise à disposition d'une bande rythmo exploitable
- Conserver un contrôle qualité humain total (rien n'est imposé, tout est modifiable)
- Respecter les conventions typographiques propres à chaque studio (personnalisables)
- Pouvoir traiter plusieurs projets en parallèle avec plusieurs intervenants
-

Exporter vers les logiciels de calligraphie et de projection existants du marché • Tracer les modifications (qui a modifié quoi, quand) pour la validation qualité

Besoins des adaptateurs et directeurs artistiques

- Visualiser immédiatement la synchronisation labiale sur la vidéo
- Identifier automatiquement les répliques trop longues ou trop courtes par rapport au temps de bouche disponible
- Repérer les respirations et points d'appui pour un jeu d'acteur naturel
- Comparer plusieurs versions d'une même réplique

Besoins non-fonctionnels

- Sécurité et confidentialité des contenus (souvent sous embargo avant diffusion mondiale)
- Performance : traitement d'une vidéo de 20 minutes en moins de 10 minutes
- Disponibilité : SaaS accessible 24/7, faible latence d'édition
- Scalabilité : montée en charge lors de pics de production (rentrées de saison)
- Interopérabilité avec l'écosystème existant (formats, métadonnées EBU/SMPTE)

2.4 Exigences réglementaires et conventionnelles

- **Codes du doublage** — crochets d'entrée [et de sortie], double-crochet pour l'ouverture de bouche silencieuse (OFF/IN/ON), italique pour les voix off/téléphone, MAJUSCULES pour les cris, parenthèses pour les indications de jeu — configurables par studio via un système de « profils typographiques ».
- **Confidentialité des contenus** — nombreux contenus soumis à un embargo strict (fuites) : chiffrement au repos et en transit, filigrane (watermark) dynamique sur les previews, journalisation des accès (obligatoire pour certains contrats studios/majors).
- **Accessibilité** — interface conforme RGAA/WCAG 2.1 niveau AA pour l'éditeur (navigation clavier, contrastes, lecteurs d'écran pour les métadonnées).
- **RGPD** — traitement de données personnelles (voix des comédiens, données utilisateurs) : base légale, minimisation, droit à l'effacement, hébergement UE recommandé.

3. Personas

3.1 Camille — Calligraphe rythmo senior

Profil : 38 ans, 12 ans d'expérience en studio, experte des logiciels de calligraphie traditionnels. Travaille à la mission pour plusieurs studios parisiens.

Objectifs

- Produire une bande rythmo de qualité irréprochable plus rapidement
- Garder la main sur les finitions artistiques (le « geste » du calligraphe)
- Pouvoir travailler sur plusieurs projets en simultané

Frustrations actuelles

- Temps considérable perdu sur le repérage mécanique (silences, respirations)
- Outils actuels peu ergonomiques, souvent hérités des années 1990-2000
- Aucune assistance automatique à la synchronisation labiale

Fonctionnalités clés attendues

- Bande rythmo pré-générée éditable finement, image par image
- Raccourcis clavier professionnels
- Visualisation de l'onde sonore et des phonèmes alignés

3.2 Julien — Directeur artistique de doublage

Profil : 45 ans, dirige les sessions d'enregistrement en studio, valide la qualité de l'adaptation et du jeu d'acteur.

Objectifs

- Disposer rapidement d'une bande exploitable pour lancer les sessions d'enregistrement
- Vérifier la cohérence émotion/texte avant la session
- Collaborer avec les adaptateurs à distance

Frustrations actuelles

- Délais de production serrés imposés par les diffuseurs
- Difficulté à visualiser les problèmes de calage avant d'être en cabine
- Échanges de fichiers non versionnés par email

Fonctionnalités clés attendues

- Tableau de bord projet avec statut d'avancement
- Système de commentaires et validation collaborative
- Prévisualisation vidéo + bande en un clic

3.3 Sofia — Adaptatrice / auteure de doublage

Profil : 31 ans, traduit et adapte des scripts pour le doublage FR, travaille en freelance pour plusieurs studios européens.

Objectifs

- Ajuster le texte pour respecter le temps de bouche disponible
- Recevoir une assistance sur le rythme et la longueur des répliques
- Travailler hors ligne puis synchroniser

Frustrations actuelles

- Pas de retour automatique sur le calibrage syllabique du texte
- Va-et-vient permanent entre le texte, l'audio et l'image
- Aucun outil ne détecte les respirations pour caler les relances

Fonctionnalités clés attendues

- Indicateur de calibrage syllabes/secondes en temps réel
- Détection automatique des respirations et silences exploitables
- Export/production de scripts annotés

3.4 Marc — Chef de projet / gérant de studio de doublage

Profil : 50 ans, dirige un studio de taille moyenne (15 collaborateurs), gère plannings, budgets et relation client (diffuseurs, plateformes SVOD).

Objectifs

- Réduire les coûts de production sans dégrader la qualité
- Suivre l'avancement de tous les projets en un tableau de bord
- Facturer précisément le temps machine et humain

Frustrations actuelles

- Manque de visibilité temps réel sur l'avancement des bandes rythmo
- Dépendance à un petit nombre de calligraphes expérimentés
- Difficulté à onboarder de nouveaux talents rapidement

Fonctionnalités clés attendues

- Gestion multi-utilisateurs et multi-projets avec rôles
- Statistiques d'usage et de performance par projet
- Facturation/abonnement SaaS clair (voir modèle économique)

4. Benchmark des Logiciels Existants

Le marché de la préparation de bande rythme est occupé par des outils historiques, souvent développés par et pour des studios spécifiques, avec une intégration IA quasi inexistante. Le tableau ci-dessous synthétise les principales solutions identifiées et leur positionnement.

Solution	Type	Points forts	Limites
Cavena / Dubbing Suite	Logiciel installé, historique	Standard historique de l'industrie, formats reconnus	Interface datée, pas d'IA, coûteux, Windows only
Rythmo-Editor (outils internes studios)	Outils propriétaires	Adaptés aux workflows internes	Non interopérables, non maintenus, pas de cloud
VoQuent / plugins Avid/Pro Tools	Plugin DAW	Intégré à la chaîne de post-production audio	Pensé montage son, pas calligraphie rythme dédiée
Solutions de sous-titrage (Ooona, EZTitles)	SaaS sous-titrage	Cloud, collaboratif, transcription IA intégrée	Pensé sous-titrage (SRT/VTT), pas les codes du doublage ni la bande rythme
Outils maison + Excel/Word	Bricolage	Gratuit, flexible	Aucune automatisation, forte dépendance humaine

4.1 Analyse des solutions adjacentes en transcription/IA

Outil	Fonction	Forces	Limites pour ce projet
Whisper (OpenAI) / Whisper Large v3	Transcription	Très bonne précision multilingue, open-source, exécutable localement	Horodatage natif au niveau segment, pas mot-à-mot précis seul
WhisperX	Alignement mot-à-mot	Alignement forcé précis (phonème/mot), rapide, open-source	Nécessite modèle d'alignement par langue, dépendance GPU

pyannote.audio	Diarisation	État de l'art open-source pour la séparation de locuteurs	Sensible au bruit de fond, nécessite tuning
Deepgram	Transcription cloud	Très rapide, API robuste, bon support streaming	Coût à l'usage, dépendance à un fournisseur tiers, langue FR moins fine que Whisper v3 sur certains accents
ElevenLabs (Scribe)	Transcription + alignement	Bonne précision, horodatage mot par mot natif	Solution propriétaire, coût, disponibilité API variable
MFA (Montreal Forced Aligner)	Alignement phonétique	Références académiques solides, contrôle fin du dictionnaire phonétique	Plus lent, configuration experte requise

4.2 Positionnement de RythmoAI

RythmoAI se positionne comme le premier SaaS combinant (1) une chaîne IA de bout en bout spécialisée doublage (transcription + diarisation + prosodie + détection de synchronisation labiale) et (2) un éditeur de bande rythmo professionnel respectant les codes du métier, avec (3) une interopérabilité complète vers les formats existants (pour ne pas rompre les workflows des studios équipés d'outils historiques).

Avantage concurrentiel clé

Aucun concurrent identifié ne combine aujourd'hui une chaîne IA spécialisée « bande rythmo » (respirations, synchronisation labiale, codes typographiques du doublage) avec un export natif vers les formats historiques du secteur (Cavena, EBU-STL étendu).

5. Architecture Globale

5.1 Vue d'ensemble

L'architecture retenue est une architecture SaaS multi-tenant en microservices découplés, orchestrée autour d'une API REST FastAPI centrale, d'un frontend web statique HTML/CSS/JS natif, et d'un pipeline de traitement IA asynchrone exécuté sur des workers dédiés (avec ou sans GPU). Cette architecture garantit la scalabilité horizontale des tâches lourdes (transcription, alignement, rendu vidéo) indépendamment du serveur web principal.

5.2 Schéma logique des composants

Couche	Composants	Rôle
Présentation	SPA HTML5/CSS3/JS natif, Video.js, WaveSurfer.js	Interface utilisateur, éditeur de bande rythmo, lecteur A/V synchronisé
API / Application	FastAPI (Python 3.13+), Pydantic v2, SQLAlchemy 2.0	Logique métier, contrats API REST/OpenAPI, authentification, orchestration
Traitement asynchrone	Celery <i>Dramatiq</i> + <i>Redis (broker)</i> , <i>workers GPUCPU</i>	Extraction audio, transcription, diarisation, prosodie, rendu
Intelligence artificielle	Whisper Large v3, WhisperX, pyannote.audio, modèles prosodiques	Analyse audio/vidéo, génération assistée de la bande rythmo
Données	PostgreSQL, Redis (cache/queue), stockage objet (S3-compatible)	Persistance structurée, cache, fichiers médias volumineux
Infrastructure	Services Windows natifs (NSSM), Nginx pour Windows, scripts .bat/.ps1, CDN	Gestion des processus, reverse proxy, scalabilité, diffusion statique — sans conteneurisation

5.3 Diagramme de flux (texte structuré)

Le schéma ci-dessous décrit le parcours d'un fichier vidéo depuis l'import jusqu'à l'export de la bande rythmo :

1. L'utilisateur importe une vidéo via le frontend (upload direct ou upload multipart vers stockage objet via URL pré-signée).
2. Le backend FastAPI crée un enregistrement « Projet » et déclenche une tâche asynchrone (Celery) d'extraction audio (FFmpeg).
3. La tâche d'extraction produit une piste audio normalisée (WAV 16 kHz mono) stockée dans le bucket de traitement.
4. Le pipeline IA est déclenché en chaîne : transcription (Whisper), diarisation (pyannote), alignement mot-à-mot (WhisperX), analyse prosodique et détection des respirations, détection d'émotions/intentions.
5. Les résultats structurés (JSON) sont persistés en base et un moteur de règles génère automatiquement une première version de la bande rythmo (segmentation en répliques, codes typographiques).
6. Le frontend est notifié (WebSocket / polling) de la disponibilité de la bande générée et charge l'éditeur synchronisé avec la vidéo.
7. L'utilisateur édite manuellement la bande (déplacement, fusion, découpe, retouche des codes).
8. L'utilisateur exporte la bande dans le ou les formats souhaités ; le fichier est généré à la demande et téléchargeable, ou archivé dans l'espace projet.

5.4 Principes directeurs d'architecture

- **Asynchrone par défaut** — toute opération de traitement lourd (> 2 secondes) est déléguée à une file de tâches, jamais exécutée dans le cycle requête/réponse HTTP.
- **Statelessness de l'API** — les instances FastAPI sont sans état, la session est portée par un token JWT, permettant une scalabilité horizontale simple derrière un load balancer.
- **Séparation stricte calcul / stockage** — les workers IA n'accèdent jamais directement à la base applicative ; ils communiquent via l'API interne et le stockage objet.
- **Idempotence** — chaque tâche de pipeline est ré-exécutable sans effet de bord (reprise sur erreur, retry automatique avec backoff exponentiel).
- **Observabilité** — chaque étape du pipeline émet des métriques et logs structurés (voir chapitre Performance) pour un diagnostic rapide.

- **Modularité IA** — chaque brique IA (transcription, diarisation, prosodie...) est encapsulée derrière une interface interne stable, permettant de remplacer un modèle par un autre sans impact sur le reste du pipeline.

6. Architecture Backend

6.1 Stack technique retenue

Composant	Technologie	Rôle
Framework API	FastAPI (Python 3.13+)	Endpoints REST, validation Pydantic v2, documentation OpenAPI/Swagger auto-générée
Serveur ASGI	Uvicorn + Gunicorn (workers Uvicorn)	Serveur d'application asynchrone en production
ORM / migrations	SQLAlchemy 2.0 (async) + Alembic	Mapping objet-relationnel, gestion des migrations de schéma versionnées
Base de données	PostgreSQL 16+	Persistance relationnelle principale (multi-tenant)
Cache / broker	Redis-compatible — Memurai (build Windows natif, service Windows) en production, Redis natif accepté en développement via WSL2 si besoin	Cache applicatif, file de messages Celery, sessions, rate limiting
File de tâches	Celery (+ Flower pour supervision) ou Dramatiq	Orchestration des tâches asynchrones du pipeline IA
Stockage fichiers	S3-compatible (AWS S3, MinIO en binaire natif Windows, OVH Object Storage)	Vidéos, audio, exports, artefacts IA
Reverse proxy	Nginx (build natif pour Windows)	Terminaison TLS, répartition de charge, service des fichiers statiques
Gestion des processus / services	NSSM (Non-Sucking Service Manager), Planificateur de tâches Windows, scripts .bat/.ps1	Exécution de l'API, des workers et des services tiers en tant que processus ou services Windows natifs — sans conteneurisation
Calcul GPU	CUDA 12+, cuDNN, PyTorch (installation native via pilotes NVIDIA Windows)	Accélération des modèles IA (Whisper, WhisperX, pyannote)

6.2 Organisation modulaire du code (Clean Architecture)

Le backend adopte une architecture en couches inspirée de la Clean Architecture / architecture hexagonale, afin de découpler la logique métier des détails d'infrastructure (base de données, IA, stockage) :

Couche	Contenu
api/ (routers)	Endpoints FastAPI, schémas Pydantic d'entrée/sortie, dépendances (auth, pagination)
core/	Configuration, sécurité (JWT, hashing), gestion des exceptions, logging structuré
domain/	Entités métier pures (Projet, Réplique, Locuteur, BandeRythmo...), règles métier indépendantes du framework
services/	Cas d'usage applicatifs (orchestration, appel aux repositories et aux tâches asynchrones)
repositories/	Accès aux données (implémentations SQLAlchemy des interfaces du domaine)
tasks/	Tâches Celery/Dramatiq : extraction audio, pipeline IA, génération de bande, export
ai/	Wrappers des modèles IA (Whisper, WhisperX, pyannote, modèles prosodiques), interface commune
infrastructure/	Adaptateurs stockage objet, Redis, envoi d'emails, webhooks

Arborescence indicative du projet

backend/

- `app/api/{projects,users,studios,transcripts,rythmo,exports,auth}.py`
- `app/core/{config.py, security.py, logging.py, exceptions.py}`
- `app/domain/{entities, value_objects, rules}`
- `app/services/{project_service.py, transcription_service.py, rythmo_generation_service.py, export_service.py}`
- `app/repositories/{project_repo.py, user_repo.py, rythmo_repo.py}`
- `app/tasks/{audio_extraction.py, transcription.py, diarization.py, prosody.py, rythmo_generation.py, export.py}`
- `app/ai/{whisper_engine.py, whisperx_aligner.py, pyannote_diarizer.py, prosody_analyzer.py, emotion_detector.py}`
- `app/models/` (modèles SQLAlchemy)
- `app/schemas/` (schémas Pydantic)

- alembic/ (migrations)
- tests/{unit,integration,e2e}

6.3 Communication temps réel

Le suivi de l'avancement du pipeline (extraction → transcription → diarisation → génération) est poussé au frontend via WebSocket (FastAPI supporte nativement les WebSockets) avec repli automatique sur polling REST toutes les 3 secondes en cas d'indisponibilité du canal temps réel (proxy d'entreprise bloquant les WebSockets, par exemple).

6.4 Gestion des erreurs et résilience

- **Retry automatique** — chaque tâche Celery critique dispose d'une politique de retry (3 tentatives, backoff exponentiel) avant passage en état « échec » notifié à l'utilisateur.
- **Circuit breaker** — pour les appels à des services IA externes optionnels (Deepgram, ElevenLabs), un circuit breaker bascule automatiquement vers le modèle local (Whisper) en cas d'indisponibilité.
- **Dead Letter Queue** — les tâches en échec définitif sont routées vers une file dédiée pour analyse manuelle sans bloquer le reste du pipeline.
- **Validation stricte** — tous les schémas d'entrée/sortie sont validés par Pydantic v2 ; toute donnée non conforme est rejetée avec un code 422 explicite.

7. Architecture Frontend

7.1 Contraintes technologiques imposées

Conformément au cahier des charges, le frontend est développé en HTML5, CSS3 et JavaScript natif (ES2022+, modules ES), sans framework lourd (React, Angular, Vue exclus). Des bibliothèques ciblées sont autorisées lorsqu'elles apportent un avantage technique réel et mesurable, sans imposer de build-system complexe ni de virtual DOM.

Bibliothèque	Usage	Justification
Video.js	Lecteur vidéo personnalisable	Player HTML5 robuste, API riche pour synchronisation frame-accurate, plugins timeline
WaveSurfer.js	Visualisation de la forme d'onde audio	Rendu waveform performant en Canvas/WebAudio, marqueurs de régions natifs (repères de répliques)
FFmpeg.wasm	Prétraitement léger côté client (optionnel)	Génération de vignettes/aperçus, découpe rapide côté navigateur sans aller-retour serveur
Web Components natifs	Composants réutilisables (éditeur de réplique, timeline)	Encapsulation sans dépendance à un framework, compatible tous navigateurs modernes
idb (IndexedDB wrapper léger)	Cache local / mode dégradé hors-ligne	Persistance locale des éditions en cours en cas de coupure réseau

7.2 Organisation du code

Le frontend est structuré en modules ES natifs, organisés par domaine fonctionnel, avec un bundler léger (Vite, utilisé uniquement en mode build/dev-server, sans framework) pour l'optimisation de production (minification, tree-shaking, cache-busting) — Vite n'introduit aucun framework JS, il reste un simple outil de build.

- src/core/ — state management maison (pattern Observable/PubSub léger), routing SPA natif (History API)
- src/components/ — Web Components (VideoPlayer, RythmoTrack, WaveformView,

ReplicaEditor, Toolbar...) • src/services/ — clients API (fetch wrapper), gestion WebSocket, cache IndexedDB

- src/pages/ — assemblage des vues (Dashboard, ProjectView, EditorView, ExportView, Settings)
- src/styles/ — CSS3 modulaire (custom properties pour le theming, CSS Grid/Flexbox, pas de préprocesseur imposé)
- src/utils/ — fonctions pures (formatage timecode SMPTE, calculs de synchronisation, debounce/throttle)

7.3 Pattern d'architecture applicative

En l'absence de framework, l'application adopte un pattern « State Container + Web Components + Custom Events » : un store JavaScript central (classe légère basée sur EventTarget) détient l'état de l'édition en cours (répliques, sélection, playhead) ; les composants s'abonnent aux changements pertinents et se re-rendent de façon ciblée (DOM diffing manuel minimal, mise à jour chirurgicale des nœuds concernés pour rester performant sans virtual DOM).

Exemple de contrat du store d'état (extrait conceptuel)

Propriété	Type	Description
currentProject	Object	Métadonnées du projet chargé
replicas[]	Array<Replica>	Liste ordonnée des répliques de la bande rythmo (source de vérité éditée)
playheadMs	Number	Position de lecture courante en millisecondes
selection	Object null	Réplique(s) actuellement sélectionnée(s) dans l'éditeur
undoStack / redoStack	Array	Pile d'annulation/rétablissement (Command Pattern)
syncStatus	Enum	idle saving saved error — statut de sauvegarde auto

7.4 Performance et compatibilité

- **Rendu de la timeline** — rendu Canvas 2D pour la piste waveform et la piste bande rythmo (haute densité de repères), avec virtualisation

(seuls les éléments visibles dans le viewport sont dessinés) pour supporter des vidéos longues (films de 2h+).

- **Chargement progressif** — streaming vidéo HTTP (Range requests), chargement paresseux (lazy loading) des vignettes et de la waveform par segments.
- **Compatibilité navigateurs** — cible Chrome/Edge/Firefox/Safari dernières versions majeures (2 dernières versions), dégradation progressive (feature detection, pas de user-agent sniffing).
- **Accessibilité** — attributs ARIA sur les composants personnalisés, navigation clavier complète de l'éditeur (raccourcis configurables), contrastes conformes WCAG 2.1 AA.
- **PWA** — manifest + service worker pour mise en cache des assets statiques et fonctionnement en mode consultation hors-ligne.

8. Architecture Intelligence Artificielle

8.1 Vue d'ensemble de la chaîne IA

La chaîne IA transforme un flux audio/vidéo brut en une structure de données riche exploitable pour la génération automatique de bande rythmo. Elle est composée de sept modules enchaînés, chacun encapsulé et remplaçable indépendamment (principe de modularité, §5.4).

#	Module	Modèle(s) retenu(s)	Sortie produite
1	Extraction & normalisation audio	FFmpeg	WAV 16 kHz mono normalisé (loudness EBU R128)
2	Transcription	Whisper Large v3 (self-hosted, GPU)	Texte horodaté par segment, multilingue
3	Alignement mot-à-mot	WhisperX (forced alignment, wav2vec2)	Timestamps précis par mot (start/end en ms)
4	Diarisation (locuteurs)	pyannote.audio 3.x	Segments attribués à un identifiant de locuteur
5	Analyse prosodique & respirations	Modèle maison (Praat/parselmouth + heuristiques) + VAD (Silero-VAD)	Courbes F0/intensité, silences classifiés (respiration <i>pause</i> coupe)
6	Émotion & intention	Modèle de classification audio (wav2vec2 fine-tuné) + analyse textuelle (NLP FR)	Étiquettes émotionnelles par réplique (neutre, colère, joie, tristesse...) et intention (question, ordre, hésitation...)
7	Synchronisation labiale (optionnel V1.5)	Détection faciale + estimation d'ouverture buccale (Mediapipe FaceMesh)	Fenêtres d'ouverture/fermeture labiale par plan

8.2 Détail des modules

8.2.1 Transcription — Whisper Large v3

- Exécution self-hosted sur GPU (via faster-whisper / CTranslate2 pour un gain de vitesse x4 par rapport à l'implémentation de référence) ●

Support multilingue natif (paramétrage de la langue cible ou détection automatique) • Découpage en chunks de 30 s avec chevauchement pour limiter les pertes de contexte en début/fin de segment • Sortie enrichie : texte, langue détectée, score de confiance par segment

8.2.2 Alignement mot-à-mot — WhisperX

- Ré-alignement forcé du texte Whisper sur le signal audio via un modèle wav2vec2 phonétique par langue
- Production de timestamps début/fin pour chaque mot individuel, prérequis indispensable à un tracé de bande rythmo fidèle
- Alternative de secours : Montreal Forced Aligner (MFA) pour les langues/accents non couverts par WhisperX

8.2.3 Diarisation — pyannote.audio

- Pipeline de segmentation + clustering de locuteurs (pyannote/speaker-diarization-3.1 ou version supérieure disponible)
- Fusion avec les résultats de transcription pour associer chaque mot à un locuteur
- Interface de correction manuelle en cas de confusion (voix similaires, chevauchements de parole)

8.2.4 Détection des respirations, pauses et hésitations

Un Voice Activity Detector (Silero-VAD) segmente les zones de silence. Ces zones sont ensuite classifiées par un module heuristique + modèle léger entraîné sur des exemples annotés : respiration audible (pic d'énergie caractéristique dans les hautes fréquences avant une reprise de parole), pause syntaxique (silence > 300 ms en fin de proposition), hésitation (micro-silence < 200 ms suivi d'une reprise du même locuteur), coupe technique (silence total, absence de tout signal).

8.2.5 Détection d'émotions et d'intentions

Une double analyse est réalisée : (a) analyse acoustique (prosodie, timbre, énergie) via un modèle de classification audio pour détecter l'émotion perçue (neutre, joie, colère, tristesse, peur, surprise), et (b) analyse textuelle du contenu transcrit (NLP) pour détecter l'intention de la réplique (affirmation, question, ordre, hésitation, exclamation). Ces informations sont fournies à titre indicatif à l'adaptateur et au calligraphe — elles n'affectent jamais

automatiquement le texte, seulement les codes typographiques suggérés (ex. majuscules suggérées pour un cri détecté, italique suggéré pour une voix off/téléphone détectée).

8.2.6 Synchronisation labiale (détection d'ouverture de bouche)

Pour les plans où le visage du personnage est visible, un détecteur de repères faciaux (Mediapipe FaceMesh ou équivalent) mesure l'ouverture labiale image par image et produit une courbe d'activité labiale. Cette courbe est comparée à l'énergie du signal vocal pour fiabiliser le calage des crochets d'entrée/sortie de bande rythmo, en particulier sur les gros plans. Ce module est identifié comme fonctionnalité V1.5 (voir Roadmap) car il nécessite un travail d'entraînement/qualification spécifique plus long.

8.3 Moteur de génération automatique de la bande rythmo

Une fois les analyses IA disponibles, un moteur de règles métier (rule engine) — développé en Python, indépendant des modèles IA — assemble une première proposition de bande rythmo :

9. Segmentation des mots alignés en répliques cohérentes (regroupement par locuteur, par limite de silence significatif, par limite syntaxique détectée)
10. Calcul de la durée disponible par réplique (écart entre timestamp de début et de fin, ajusté des marges de sécurité configurables)
11. Application des codes typographiques selon le profil du studio (italique, majuscules, crochets) à partir des signaux émotion/intention/diarisation
12. Insertion des marqueurs de respiration détectés comme points d'appui visuels sur la bande
13. Calcul d'un score de confiance par réplique, affiché à l'utilisateur pour prioriser la relecture humaine (une réplique à faible confiance est surlignée)

8.4 Infrastructure de calcul IA

- **GPU** — workers dédiés avec GPU NVIDIA (A10G/L4 en cloud, ou équivalent on-premise), CUDA 12+, cuDNN ; bascule CPU automatique (dégradée en vitesse) si aucun GPU n'est disponible.

- **Quantification** — usage de modèles quantifiés (int8/float16 via CTranslate2) pour réduire l'empreinte mémoire GPU et augmenter le débit de traitement.
- **Traitement par lots** — file d'attente Celery dédiée aux tâches GPU avec limitation de concurrence par carte pour éviter les OOM (Out Of Memory).
- **Cache des modèles** — modèles préchargés en mémoire dans des workers persistants (pas de rechargement à chaque tâche) pour minimiser la latence de démarrage.
- **Versionning des modèles** — chaque résultat IA est associé à la version exacte du modèle utilisé (traçabilité, reproductibilité, réentraînement futur).

8.5 Amélioration continue

Un mécanisme de feedback loop est prévu : chaque correction manuelle effectuée par un utilisateur (recalage d'un mot, correction d'un locuteur, changement de code typographique) est journalisée de façon anonymisée (avec consentement contractuel studio) afin de constituer un corpus d'entraînement pour affiner les modèles heuristiques (prosodie, émotion) au fil du temps, sans jamais réentraîner les modèles de fondation tiers (Whisper, pyannote) hors du cadre de leurs licences respectives.

9. Base de Données

9.1 Choix technologique

PostgreSQL 16+ est retenu comme système de gestion de base de données relationnelle principal, pour sa robustesse transactionnelle (ACID), son support natif du JSONB (stockage flexible des résultats IA bruts), de la recherche plein texte, et son excellente intégration avec SQLAlchemy 2.0 (mode asynchrone via asyncpg). Redis complète le dispositif pour le cache, les sessions et les files de tâches.

9.2 Modèle de données — Entités principales

Entité	Description
Studio	Organisation cliente (tenant) — nom, plan d'abonnement, profils typographiques personnalisés, quotas
User	Utilisateur — email, mot de passe hashé, rôle global, appartenance à un ou plusieurs studios
StudioMembership	Table de jonction Studio ↔ User avec rôle contextuel (admin, chef de projet, adaptateur, calligraphe, DA, invité)
Project	Projet de doublage — titre, langue source/cible, statut, studio propriétaire, vidéo associée
MediaAsset	Fichier média importé (vidéo/audio) — chemin stockage objet, métadonnées techniques (durée, codec, fps, résolution)
TranscriptSegment	Segment de transcription brute issu de Whisper — texte, start/end, langue, score de confiance
Word	Mot individuel aligné (issu de WhisperX) — texte, start_ms, end_ms, segment parent, locuteur
Speaker	Locuteur détecté par diarisation — identifiant, label (renommable par l'utilisateur), couleur d'affichage
SilenceEvent	Événement de silence classifié — type (respiration/pause/hésitation/coupe), start_ms, end_ms, confiance
EmotionTag	Étiquette émotionnelle/intention associée à un segment — type, score, source (audio/texte)
RythmoBand	Bande rythmo d'un projet — version, statut (brouillon/validée), profil typographique appliqué

Replica	Réplique individuelle de la bande — texte affiché, start_ms, end_ms, locuteur, codes typographiques, ordre
ReplicaHistory	Historique des modifications d'une réplique (audit trail) — auteur, date, diff avant/après
TypographicProfile	Profil de règles typographiques par studio — codes, conventions, seuils de calibrage
ExportJob	Tâche d'export — format cible, statut, fichier résultat, date
Comment	Commentaire collaboratif attaché à une réplique ou à un projet
AuditLog	Journal d'audit global (connexions, actions sensibles, exports, partages)
Subscription	Abonnement SaaS du studio — plan, cycle de facturation, quotas consommés

9.3 Schéma relationnel simplifié (extrait des relations clés)

Relation	Cardinalité	Commentaire
Studio → Project	1 — N	Un studio possède plusieurs projets
Studio ↔ User (via StudioMembership)	N — N	Un utilisateur peut appartenir à plusieurs studios avec des rôles différents
Project → MediaAsset	1 — N	Un projet peut avoir plusieurs versions de média (v1, v2 corrigée...)
MediaAsset → TranscriptSegment	1 — N	Un média génère plusieurs segments de transcription
TranscriptSegment → Word	1 — N	Un segment contient plusieurs mots alignés
Project → RythmoBand	1 — N	Un projet peut avoir plusieurs versions de bande rythmo (historisation)
RythmoBand → Replica	1 — N	Une bande est composée d'un ensemble ordonné de répliques
Replica → ReplicaHistory	1 — N	Chaque modification est historisée pour audit
Replica → Speaker	N — 1	Chaque réplique est attribuée à un locuteur

9.4 Détail des colonnes — Table `replica` (exemple représentatif)

Colonne	Type	Description
id	UUID (PK)	Identifiant unique
rythmo_band_id	UUID (FK)	Référence vers la bande rythmo parente
speaker_id	UUID (FK, nullable)	Locuteur associé
text	TEXT	Contenu textuel affiché sur la bande
start_ms / end_ms	INTEGER	Bornes temporelles en millisecondes
order_index	INTEGER	Position dans la séquence (tri stable)
typo_codes	JSONB	Codes typographiques appliqués (italique, majuscules, crochets, off...)
confidence_score	NUMERIC(4,3)	Score de confiance de la génération automatique (0 à 1)
is_manually_edited	BOOLEAN	Indique si la réplique a été modifiée manuellement depuis sa génération
breath_marker	BOOLEAN	Présence d'un marqueur de respiration en début de réplique
created_at / updated_at	TIMESTAMPTZ	Horodatage de création / dernière modification

9.5 Stratégie d'indexation et de performance

- Index B-Tree sur les clés étrangères (project_id, rythmo_band_id, speaker_id) et sur order_index pour les tris de répliques
- Index GIN sur les colonnes JSONB (typo_codes, métadonnées IA brutes) pour permettre des requêtes de filtrage avancées
- Index composite (rythmo_band_id, start_ms) pour l'affichage rapide de la timeline dans l'éditeur
- Partitionnement envisagé de la table AuditLog par mois au-delà d'un volume significatif (archivage à froid après 24 mois)
- Utilisation de UUID v7 (ordonnés temporellement) comme clés primaires pour concilier unicité distribuée et performance d'indexation

9.6 Isolation multi-tenant

L'isolation des données entre studios (tenants) est assurée au niveau applicatif (filtrage systématique par studio_id dans chaque requête, imposé par la couche repository) et renforcée par des Row-Level Security (RLS)

PostgreSQL sur les tables sensibles, offrant une double barrière de sécurité contre les fuites de données inter-studios en cas de bug applicatif.

9.7 Migrations et gouvernance de schéma

- Toute évolution du schéma passe par une migration Alembic versionnée, revue en pull request
- Migrations rétro-compatibles par défaut (expand/contract pattern) pour permettre des déploiements sans interruption de service
- Jeu de données de test (fixtures) maintenu en parallèle pour les environnements de développement et de recette

10. API REST

10.1 Principes de conception

- API REST versionnée (préfixe *apiv1/*), documentation OpenAPI générée automatiquement par FastAPI (Swagger UI + Redoc)
- Authentification par JWT (access token courte durée + refresh token), transmise via header Authorization: Bearer
- Réponses au format JSON, pagination par curseur pour les collections volumineuses (transcripts, répliques)
- Codes HTTP standards stricts (200/201/204, 400/401/403/404/409/422, 500/503)
- Gestion des erreurs uniformisée (schéma d'erreur commun : code, message, details, request_id)
- Rate limiting par utilisateur/studio (protection contre les abus, quotas d'usage IA)

10.2 Cartographie des ressources principales

Ressource	Endpoints principaux
Auth	POST <i>authregister</i> , POST <i>authlogin</i> , POST <i>authrefresh</i> , POST <i>authlogout</i> , POST <i>authmfa/verify</i>
Users	GET/PATCH <i>usersme</i> , GET <i>studios{id}/users</i> , POST <i>studios{id}usersinvite</i>
Studios	GET/POST <i>studios</i> , GET/PATCH <i>studios{id}</i> , GET/PATCH <i>studios{id}/typographic-profiles</i>
Projects	GET/POST <i>projects</i> , GET/PATCH/DELETE <i>projects{id}</i> , GET <i>projects{id}/activity</i>
Media	POST <i>projects{id}mediaupload-url</i> , POST <i>projects{id}media{id}/confirm</i> , GET <i>media{id}</i>
Pipeline IA	POST <i>projects{id}/pipeline/start</i> , GET <i>projects{id}/pipeline/status</i> , WS <i>/wsprojects{id}/pipeline</i>
Transcript	GET <i>projects{id}/transcript</i> , PATCH <i>transcriptsegments/{id}</i> , PATCH <i>transcriptwords/{id}</i>
Speakers	GET <i>projects{id}/speakers</i> , PATCH <i>speakers{id}</i> (renommage, fusion)
Rythmo Band	GET <i>projects{id}/rythmo</i> , POST <i>projects{id}rythmogenerate</i> , GET/POST <i>rythmo{id}/replicas</i>

Replica	PATCH/DELETE <i>replicas</i> {id}, POST <i>replicas</i> {id}/split, POST <i>replicas</i> merge
Export	POST <i>projects</i> {id}/exports, GET <i>exports</i> {id}, GET <i>exports</i> {id}/download
Comments	GET/POST <i>replicas</i> {id}/comments, DELETE <i>comments</i> {id}
Billing	GET <i>studios</i> {id}/subscription, GET <i>studios</i> {id}/usage

10.3 Exemple de contrat détaillé — Génération de la bande rythmo

POST *apiv1projects*{project_id}*rythmogenerate*

Élément	Détail
Description	Déclenche la génération automatique (ou régénération) d'une bande rythmo à partir des résultats du pipeline IA disponibles
Auth requise	Bearer JWT — rôle minimum : adaptateur ou supérieur sur le studio propriétaire
Corps de requête	{ "typographic_profile_id": UUID, "regenerate": boolean, "segmentation_strategy": "auto speaker silence" }
Réponse 202	{ "job_id": UUID, "status": "queued" } — traitement asynchrone, suivi via WebSocket ou GET <i>rythmojobs</i> /{job_id}
Réponse 409	Retournée si le pipeline IA préalable (transcription/diarisation) n'est pas encore terminé
Réponse 422	Corps de requête invalide (profil typographique inexistant, stratégie inconnue)

PATCH *apiv1replicas*{replica_id}

Élément	Détail
Description	Met à jour une réplique existante (texte, timing, codes typographiques, locuteur)
Corps de requête	{ "text": string?, "start_ms": int?, "end_ms": int?, "speaker_id": UUID?, "typo_codes": object? }
Comportement	Toute modification déclenche la création d'une entrée ReplicaHistory et marque is_manually_edited=true
Validation métier	start_ms < end_ms, pas de chevauchement autorisé avec la réplique suivante du même locuteur sauf option overlap_allowed
Réponse 200	Réplique mise à jour, incluant le score de confiance recalculé si

10.4 Sécurité des endpoints

- Contrôle d'accès basé sur les rôles (RBAC) appliqué systématiquement via des dépendances FastAPI réutilisables (`require_role(...)`)
- Vérification systématique de l'appartenance de la ressource au studio de l'utilisateur authentifié (protection anti-IDOR)
- Endpoints d'upload protégés par URLs pré-signées à durée de vie limitée (10 minutes) plutôt que par upload direct au serveur applicatif
- Validation stricte des types de fichiers acceptés (whitelist d'extensions et vérification de signature binaire, pas seulement d'extension)

10.5 Documentation et contrat

La documentation OpenAPI 3.1 générée automatiquement par FastAPI est publiée sur *docs* (*Swagger UI, environnements non-production*) et *redoc*, et exportée en JSON pour intégration dans les pipelines de tests de contrat (contract testing) et la génération de SDK clients si nécessaire (studios souhaitant scripter leurs intégrations).

11. Pipeline Vidéo

11.1 Import et validation

- Formats acceptés en entrée : MP4 (H.264/H.265), MOV (ProRes), MXF, AVI, MKV — validation par inspection des flux (ffprobe), pas seulement par extension
- Taille maximale configurable par plan d'abonnement (ex. 20 Go en plan Studio, upload multipart résumable)
- Upload direct vers le stockage objet via URL pré-signée (le fichier ne transite jamais par le serveur applicatif FastAPI, seul le worker de traitement y accède)
- Vérification d'intégrité (checksum) et détection de fichiers corrompus avant déclenchement du pipeline

11.2 Normalisation et extraction

14. Extraction des métadonnées techniques (durée, fps, résolution, codec, présence de pistes audio multiples) via ffprobe
15. Extraction de la ou des pistes audio en WAV 16 kHz mono (FFmpeg), une extraction par piste si le fichier contient plusieurs langues/versions
16. Génération d'un proxy vidéo basse résolution (720p, débit réduit) pour fluidifier la lecture dans l'éditeur web sans dépendre du fichier source lourd
17. Génération d'une bande de vignettes (sprite sheet) pour la navigation rapide dans la timeline
18. Détection des changements de plan (scene detection, PySceneDetect) utile à la synchronisation labiale et à la segmentation visuelle des répliques

11.3 Diffusion et lecture synchronisée

- Diffusion du proxy vidéo en streaming HTTP progressif (Range requests) ou HLS pour les fichiers très longs (séries, films)
- Synchronisation frame-accurate entre le player vidéo (Video.js) et la piste bande rythmo (résolution SMPTE configurable : 24/25/29.97 fps)
- Gestion du timecode source (burnt-in ou métadonnée) pour garantir la correspondance avec les conventions de repérage du studio

11.4 Synchronisation labiale (module V1.5)

Le pipeline vidéo alimente, pour les plans avec visage détecté, le module de synchronisation labiale décrit au chapitre 8.2.6 : extraction d'images à cadence régulière, détection de visage, mesure de l'ouverture buccale normalisée, corrélation avec le signal audio pour fiabiliser les bornes de réplique sur les gros plans.

12. Pipeline Audio

12.1 Prétraitement du signal

- Normalisation du niveau sonore (loudness EBU R128 ou normalisation de crête) pour homogénéiser la qualité d'entrée des modèles IA
- Réduction de bruit optionnelle (denoising) configurable, désactivable si elle dégrade la précision de transcription sur certains contenus (musique de fond forte, effets)
- Détection et séparation des pistes multiples (dialogue *musique* effets) via un modèle de séparation de sources si le fichier fourni est un mixage complet (amélioration V2)

12.2 Chaîne de traitement séquentielle

Étape	Traitement	Détail technique
1	VAD (Voice Activity Detection)	Silero-VAD segmente les zones de parole vs silence, réduisant la charge de calcul en aval
2	Transcription	Whisper Large v3 sur les segments de parole détectés
3	Alignement mot-à-mot	WhisperX produit les bornes précises par mot
4	Diarisation	pyannote.audio segmente et regroupe par locuteur, fusion avec les mots alignés
5	Analyse prosodique	Extraction F0 (pitch), intensité, débit de parole (syllabes/seconde) via parselmouth (Praat)
6	Classification des silences	Modèle heuristique + léger classifieur distingue respiration <i>pause</i> hésitation / coupe
7	Détection émotion audio	Modèle wav2vec2 fine-tuné pour la classification d'émotion vocale

12.3 Analyse du rythme et de la vitesse d'élocution

Pour chaque réplique, le pipeline calcule un débit d'élocution (syllabes par seconde, estimé via un compteur syllabique du texte transcrit rapporté à la durée mesurée) comparé à des seuils de référence configurables par langue (ex. 5 à 7 syllabes/seconde en français standard). Un écart significatif est signalé à l'adaptateur comme indice de risque de désynchronisation labiale ou de diction trop rapide/lente pour le comédien.

12.4 Fiabilité et contrôle qualité automatique

- Score de confiance agrégé par réplique (moyenne pondérée des scores de transcription, d'alignement et de diarisation)
- Détection automatique des zones à faible confiance (chevauchement de locuteurs, bruit important, musique forte) signalées visuellement dans l'éditeur
- Journal d'analyse exportable pour audit qualité (rapport PDF de synthèse du traitement IA d'un projet)

13. Pipeline IA — Orchestration Complète

13.1 Orchestration Celery

L'ensemble du pipeline est orchestré comme une chaîne de tâches Celery (canvas : chain/group/chord) permettant le parallélisme là où c'est pertinent (ex. diarisation et analyse prosodique peuvent s'exécuter en parallèle une fois la normalisation audio terminée) tout en respectant les dépendances strictes (l'alignement mot-à-mot nécessite la transcription complète).

Schéma de la chaîne de tâches

19. `extract_audio` (CPU) → `normalize_audio` (CPU)
20. `transcribe` (GPU) — dépend de `normalize_audio`
21. `group( align_words (GPU), diarize (GPU), analyze_prosody (CPU) )` — exécution parallèle, dépendent de `transcribe`
22. `chord( detect_emotions (GPU/CPU), classify_silences (CPU) )` — dépendent du groupe précédent
23. `generate_rythmo_band` (CPU, règles métier) — étape finale d'assemblage
24. `notify_completion` (WebSocket + email optionnel)

13.2 Gestion des priorités et files dédiées

File Celery	Type de ressource	Tâches
queue.gpu.high	GPU, priorité haute	Transcription et alignement des projets en plan Studio/Enterprise
queue.gpu.default	GPU, priorité standard	Transcription et alignement des projets en plan standard
queue.cpu.audio	CPU	Extraction, normalisation, analyse prosodique, classification des silences
queue.cpu.export	CPU	Génération des fichiers d'export (rendu PDF, formats propriétaires)

13.3 Suivi d'avancement et expérience utilisateur

Chaque étape du pipeline met à jour un enregistrement PipelineJob (statut, pourcentage estimé, étape courante) exposé en temps réel au frontend. L'utilisateur visualise une barre de progression détaillée (« Transcription en cours — 64 % », « Détection des locuteurs... ») plutôt qu'un simple spinner, avec estimation du temps restant basée sur la durée moyenne observée par minute de programme.

13.4 Objectifs de performance du pipeline

Étape	Cible de performance (GPU dédié)	Cible de performance (CPU seul)
Extraction/normalisation audio	< 5 % de la durée du média	< 5 % de la durée du média
Transcription + alignement	≈ 15 à 25 % de la durée du média	≈ 100 à 150 % de la durée du média
Diarisation	≈ 10 à 15 % de la durée du média	≈ 40 à 60 % de la durée du média
Génération de la bande rythmo	< 10 secondes (règles métier, non-IA lourde)	< 10 secondes
Pipeline complet (vidéo de 20 min)	≈ 6 à 9 minutes	≈ 30 à 45 minutes

14. UX / UI

14.1 Principes directeurs

- **Densité maîtrisée** — l'éditeur professionnel manipule beaucoup d'information (waveform, bande rythme, vidéo, métadonnées locuteurs) : la mise en page privilégie la clarté hiérarchique plutôt que la sobriété minimaliste généraliste.
- **Clavier-first** — les utilisateurs professionnels (calligraphes) travaillent majoritairement au clavier ; chaque action fréquente dispose d'un raccourci, affiché et personnalisable.
- **Confiance dans l'IA sans dépendance** — les éléments générés automatiquement sont visuellement distingués (halo de couleur selon le score de confiance) des éléments validés/édités manuellement, sans jamais bloquer l'édition libre.
- **Feedback immédiat** — toute action d'édition se reflète instantanément dans la bande et dans la vidéo (lecture synchronisée au point d'édition).

14.2 Écrans principaux

14.2.1 Tableau de bord (Dashboard)

- Vue synthétique des projets du studio (statut, avancement pipeline, dernière modification, intervenants)
- Filtres par statut (en traitement, prêt pour édition, en relecture, validé, livré)
- Indicateurs studio (temps moyen de traitement, volume traité dans le mois, quota d'usage restant)

14.2.2 Écran d'import de projet

- Zone de dépôt (drag & drop) avec barre de progression d'upload résumable
- Sélection de la langue source/cible, du profil typographique du studio, des options de pipeline (activation de la synchronisation labiale, réduction de bruit)

14.2.3 Éditeur de bande rythme (écran central)

L'écran est structuré en quatre zones synchronisées, empilées verticalement, conformément à la structure attendue par les professionnels du secteur :

Zone	Contenu et interactions
Zone vidéo (haut)	Lecteur Video.js, contrôles standards, overlay optionnel des repères labiaux (mode debug/QC)
Zone waveform	Forme d'onde WaveSurfer.js avec régions colorées par locuteur, marqueurs de respiration, zoom temporel
Zone bande rythmo	Piste de répliques éditables : glisser pour déplacer, poignées pour redimensionner, double-clic pour éditer le texte, clic droit pour les codes typographiques
Zone transcription/liste	Liste linéaire des répliques (vue alternative « script »), utile pour la relecture séquentielle et la recherche textuelle

14.2.4 Panneau latéral contextuel

- Détails de la réplique sélectionnée (locuteur, timing précis, score de confiance, historique des modifications)
- Fil de commentaires collaboratifs attaché à la réplique
- Suggestions IA (ex. « débit élevé détecté », « respiration non exploitée disponible à 00:12:34 »)

14.2.5 Écran d'export

- Sélection du ou des formats de sortie, aperçu avant génération
- Historique des exports précédents avec re-téléchargement

14.3 Système de design

- Palette sombre par défaut (confort en environnement de studio faiblement éclairé), thème clair disponible
- Typographie monospaced pour les zones de timecode (lisibilité et alignement des chiffres), typographie humaniste pour le texte des répliques
- Composants réutilisables documentés (bouton, champ, badge de statut, tag locuteur, indicateur de confiance) formant une mini bibliothèque de composants Web Components internes
- Codage couleur cohérent : un identifiant de couleur unique par locuteur, réutilisé dans la waveform, la bande et la liste

14.4 Raccourcis clavier de référence (extrait)

Raccourci	Action	Contexte
Espace	Lecture / pause	Global
I / O	Poser le point d'entrée / sortie de la réplique sélectionnée	Éditeur bande rythmo
Ctrl+Entrée	Valider et passer à la réplique suivante	Édition de texte
Ctrl+Maj+F	Fusionner avec la réplique suivante	Sélection réplique
Ctrl+Maj+S	Scinder la réplique au point de lecture	Édition
Ctrl+Z / Ctrl+Y	Annuler / rétablir	Global

14.5 Tests utilisateurs et validation

Le processus de conception prévoit des sessions de tests utilisateurs avec des calligraphes et adaptateurs professionnels dès les maquettes basse fidélité (papier/Figma), puis sur prototype fonctionnel (protocole think-aloud), afin de valider l'adéquation avec les usages métiers réels avant le développement final de chaque écran majeur.

15. Sécurité et Authentification

15.1 Enjeux spécifiques au domaine

Les contenus traités (films, séries, épisodes non diffusés) sont fréquemment soumis à des embargos stricts de la part des studios/majors, avec des clauses contractuelles de sécurité renforcée (chiffrement, watermarking, journalisation, restrictions géographiques). La sécurité n'est donc pas une simple bonne pratique mais un prérequis commercial pour signer des contrats avec les grands studios.

15.2 Authentification

- **Protocole** — OAuth2 / OpenID Connect avec JWT (access token 15 min, refresh token 7 jours, rotation à chaque utilisation).
- **MFA** — authentification à deux facteurs (TOTP) obligatoire pour les comptes administrateurs de studio, optionnelle et encouragée pour les autres rôles.
- **SSO Enterprise** — support SAML 2.0 / OIDC pour les studios de grande taille disposant d'un fournisseur d'identité (Azure AD, Okta, Google Workspace) — fonctionnalité plan Enterprise.
- **Mots de passe** — hashage Argon2id, politique de complexité configurable, détection de mots de passe compromis (vérification contre des bases de fuites connues type Have I Been Pwned range API).
- **Sessions** — possibilité de révocation immédiate de toutes les sessions actives d'un utilisateur (compte compromis, départ d'un collaborateur).

15.3 Autorisation (RBAC)

Rôle studio	Permissions
Owner / Admin studio	Gestion complète (utilisateurs, facturation, profils typographiques, tous les projets)
Chef de projet	Création/gestion des projets, invitation de collaborateurs, validation finale, export

Directeur artistique	Relecture, commentaires, validation qualité, édition de la bande rythmo
Adaptateur	Édition du texte des répliques, consultation des analyses IA, commentaires
Calligraphe rythmo	Édition complète de la bande rythmo (timing, codes), export
Invité / Client (lecture seule)	Consultation et commentaires uniquement, aucune modification, export limité (PDF preview filigrané)

15.4 Protection des contenus

- **Chiffrement au repos** — AES-256 sur le stockage objet (chiffrement côté serveur avec gestion de clés dédiée, KMS), chiffrement des volumes de base de données.
- **Chiffrement en transit** — TLS 1.3 obligatoire sur tous les flux (frontend ↔ API, API ↔ stockage, API ↔ workers), HSTS activé.
- **Filigrane dynamique** — watermark visuel (nom d'utilisateur, date, heure) incrusté sur les aperçus vidéo et les exports PDF pour les rôles à risque (invités, clients externes), dissuasif contre la fuite.
- **URLs signées à expiration courte** — aucun accès direct et permanent aux fichiers médias ; chaque accès génère une URL pré-signée à durée de vie limitée (quelques minutes).
- **Isolation des exports** — fichiers d'export stockés dans un espace distinct avec purge automatique configurable (ex. suppression après 30 jours sauf archivage explicite).

15.5 Journalisation et audit

- Journal d'audit exhaustif des actions sensibles (connexions, exports, partages, changements de droits, téléchargements) horodaté et non modifiable (append-only)
- Conservation des logs de sécurité 12 mois minimum (ajustable selon exigences contractuelles studio)
- Alerting automatique sur comportements anormaux (téléchargements massifs, connexions depuis des géolocalisations inhabituelles, tentatives de force brute)

15.6 Conformité RGPD

- Hébergement des données en Union Européenne (choix d'un cloud provider avec régions UE, ou hébergement souverain OVHcloud/Scaleway envisageable)
- Registre des traitements, base légale identifiée (exécution du contrat pour les données professionnelles)
- Droit d'accès, de rectification, d'effacement et de portabilité exposés via un processus documenté (support puis, à terme, self-service)
- Minimisation des données personnelles collectées (les voix des comédiens ne sont traitées que pour la finalité du projet, pas de réutilisation pour l'entraînement de modèles sans consentement explicite contractualisé)
- Analyse d'impact relative à la protection des données (AIPD/DPIA) recommandée avant le lancement commercial, en particulier sur le traitement vocal biométrique potentiel (diarisation)

15.7 Sécurité applicative (OWASP)

- Revue systématique OWASP Top 10 (injection, XSS, CSRF, IDOR, SSRF) en revue de code et tests automatisés
- En-têtes de sécurité HTTP stricts (Content-Security-Policy, X-Frame-Options, Referrer-Policy) servis par Nginx
- Scan de dépendances automatisé (pip-audit, npm audit / osv-scanner) intégré au pipeline CI
- Tests d'intrusion (pentest) externes recommandés avant chaque mise en production majeure et a minima annuellement
- Bug bounty ou canal de signalement responsable (responsible disclosure) mis en place à partir de la V1 commerciale

16. Gestion des Projets, des Utilisateurs et des Studios

16.1 Gestion des projets

Cycle de vie d'un projet

Statut	Description
Créé	Projet initialisé, en attente d'upload du média
En traitement	Pipeline IA en cours d'exécution (extraction, transcription, diarisation...)
Prêt pour édition	Bande rythmo générée automatiquement, disponible dans l'éditeur
En édition	Modifications manuelles en cours par un ou plusieurs collaborateurs
En relecture	Soumis à validation du directeur artistique / chef de projet
Validé	Bande rythmo approuvée, verrouillée en écriture sauf déverrouillage explicite
Exporté / Livré	Export(s) générés et mis à disposition du studio d'enregistrement
Archivé	Projet clos, conservé en lecture seule selon la politique de rétention

- Gestion de versions de la bande rythmo (historisation complète, comparaison entre versions, retour arrière possible)
- Verrouillage optimiste sur les répliques en cours d'édition simultanée (indication visuelle « Camille édite cette réplique »)
- Système de tags et de dossiers pour organiser les projets par client, saison, diffuseur
- Recherche full-text dans les transcriptions de l'ensemble des projets d'un studio (PostgreSQL full-text search ou moteur dédié type Meilisearch/OpenSearch à l'échelle)

16.2 Gestion des utilisateurs

- Invitation par email avec lien d'activation à durée limitée, attribution du rôle dès l'invitation
- Profil utilisateur : préférences d'affichage (thème, raccourcis personnalisés), langue d'interface
- Gestion du cycle de vie du compte (désactivation, suppression conforme RGPD,

transfert de propriété des projets en cas de départ) • Vue « Mon activité » récapitulant les projets récents et les tâches assignées

16.3 Gestion des studios (multi-tenant)

- Un studio est l'unité de facturation et d'isolation des données (tenant)
- Paramétrage des profils typographiques par studio (voir §2.4 et §8.3) — plusieurs profils possibles par studio (un par diffuseur/client si nécessaire)
- Tableau de bord d'usage et de facturation (minutes de traitement IA consommées, nombre d'utilisateurs actifs, volume de stockage)
- Gestion des sous-groupes/équipes au sein d'un grand studio (ex. « Pôle jeunesse », « Pôle films ») avec droits d'accès dédiés — fonctionnalité plan Enterprise
- Politique de rétention des données configurable par studio (durée de conservation des médias sources, des exports, des logs)

16.4 Collaboration en temps réel

Plusieurs utilisateurs peuvent consulter simultanément un même projet ; l'édition concurrente sur une même réplique est gérée par verrouillage optimiste avec notification en temps réel (WebSocket) — une architecture de type CRDT (édition collaborative façon Google Docs) est identifiée comme évolution possible en V2 pour une collaboration réellement simultanée caractère par caractère, mais n'est pas requise pour le MVP où le verrouillage par réplique suffit à l'usage métier réel (les répliques sont rarement éditées à plusieurs en simultané dans le processus actuel).

17. Performance

17.1 Objectifs de performance (SLO)

Indicateur	Cible	Mesure
Temps de chargement initial de l'éditeur	< 2,5 s (P75)	Web Vitals (LCP)
Latence d'interaction dans l'éditeur (déplacer une réplique)	< 100 ms	Trace frontend (RUM)
Latence API (endpoints CRUD standards)	< 200 ms (P95)	APM backend
Traitement pipeline IA (vidéo 20 min, GPU dédié)	< 10 min	Durée moyenne de job Celery
Disponibilité plateforme	≥ 99,5 % mensuel	Monitoring uptime
Temps de génération d'un export PDF (bande de 20 min)	< 15 s	Durée de tâche d'export

17.2 Stratégies de performance backend

- Requêtes SQL optimisées (index ciblés, eager loading maîtrisé pour éviter le N+1 via SQLAlchemy selectinload/joinedload)
- Cache Redis pour les données peu volatiles (profils typographiques, métadonnées de référence) avec invalidation ciblée
- Pagination systématique des collections volumineuses (répliques, mots, logs) avec curseur plutôt qu'offset pour les grands volumes
- Connexions base de données poolées (pgbouncer en mode transaction pour absorber les pics de connexions asynchrones)
- Compression des réponses HTTP (gzip/brotli) via Nginx

17.3 Stratégies de performance frontend

- Rendu Canvas virtualisé pour la timeline (seuls les éléments visibles sont dessinés, recalcul incrémental au scroll/zoom)
- Debounce/throttle sur les événements haute fréquence (déplacement de curseur, redimensionnement de réplique)
- Sauvegarde automatique différée (auto-save toutes les 3 secondes d'inactivité, avec indicateur de statut) plutôt qu'à chaque frappe
- Chargement progressif de la waveform par

segments (streaming des données d'amplitude au fur et à mesure du scroll temporel) • Mise en cache locale (IndexedDB) de l'état d'édition pour tolérer les micro-coupures réseau sans perte de travail

17.4 Scalabilité du pipeline IA

- Autoscaling horizontal des workers GPU en fonction de la profondeur de la file d'attente (métrique Celery/Redis exposée à l'orchestrateur)
- Priorisation des jobs par plan d'abonnement (SLA différencié Studio/Enterprise vs plan standard)
- Traitement par batch pour optimiser l'utilisation GPU (regroupement de segments courts de plusieurs projets sur un même passage modèle lorsque la charge le permet)
- Limitation du nombre de jobs concurrents par GPU pour éviter la contention mémoire (OOM), avec file d'attente équitable (fair queueing) entre studios

17.5 Tests de charge

- Tests de charge API réguliers (k6 ou Locust) simulant des scénarios réalistes (imports simultanés, édition concurrente multi-utilisateurs)
- Tests de charge dédiés au pipeline IA (simulation de pics de soumission en début de saison de production)
- Budgets de performance (performance budgets) intégrés en CI pour le frontend (taille des bundles, temps de rendu)

18. Infrastructure et Déploiement Windows (sans Docker)

Contrainte de déploiement

L'application ne repose sur aucune conteneurisation (Docker, Kubernetes, etc.). Tous les composants (API, workers, base de données, cache, reverse proxy) s'installent et s'exécutent en processus natifs sur Windows, avec un démarrage/arrêt simplifiés via des scripts .bat et .ps1 fournis à la racine du projet. Cette approche facilite l'installation on-premise directement sur les postes ou serveurs des studios, sans dépendance à Docker Desktop (dont la licence commerciale est par ailleurs payante pour les entreprises de taille moyenne).

18.1 Choix d'hébergement

Deux modes de déploiement sont pris en charge par la même base applicative, sans aucune divergence de code : (1) un poste ou serveur Windows on-premise chez le studio, pour les clients exigeant une isolation maximale de leurs contenus sous embargo ; (2) un serveur Windows distant (VM Windows Server chez un hébergeur cloud) pour le mode SaaS mutualisé. Dans les deux cas, l'installation et le lancement suivent exactement la même procédure scriptée.

Option	Avantages	Contraintes
Poste/serveur on-premise (studio)	Isolation totale des contenus, conformité contractuelle maximale, pas de coût cloud récurrent	Maintenance et disponibilité à la charge du studio, GPU à acquérir localement
VM Windows Server — Azure	Bonne intégration GPU (séries NC/NV), outillage de supervision natif Windows	Coût plus élevé, dépendance à un acteur non-européen selon la région choisie
VM Windows Server — OVHcloud / autre hébergeur UE	Souveraineté des données, tarification prévisible	Offre GPU sous Windows Server plus restreinte, à vérifier au cas par cas

18.2 Prérequis logiciels sur la machine hôte

Le script d'installation (§18.4) vérifie automatiquement la présence de chaque prérequis et guide l'utilisateur pour l'installation des éléments manquants (lien direct ou installation silencieuse via winget lorsque c'est possible).

Prérequis	Version cible	Rôle
Windows	10/11 ou Windows Server 2019+	Système d'exploitation cible
Python	3.13+ (installateur officiel Windows)	Exécution du backend FastAPI et des workers
Node.js	20 LTS+	Build des assets frontend (Vite) — build uniquement, aucun framework runtime
PostgreSQL	16+ (installateur EDB pour Windows)	Base de données applicative
Memurai (ou Redis compatible Windows)	Dernière version stable	Cache, sessions, broker de tâches Celery
FFmpeg	Build Windows (essentials/full)	Extraction et transcodage audio/vidéo
Pilotes NVIDIA + CUDA Toolkit	CUDA 12+ (si GPU disponible)	Accélération des modèles IA
NSSM	Dernière version	Enregistrement optionnel des services en tant que services Windows persistants
Nginx pour Windows	Build stable officiel	Reverse proxy et service des fichiers statiques du frontend

18.3 Architecture de déploiement par processus natifs

En l'absence de conteneurisation, chaque composant applicatif s'exécute comme un processus Windows autonome, soit lancé manuellement par les scripts de démarrage (poste de développement), soit enregistré comme service Windows via NSSM pour un redémarrage automatique en production (serveur).

Processus	Commande sous-jacente	Mode d'exécution recommandé
API FastAPI	<code>uvicorn app.main:app --host 0.0.0.0 --port 8000</code>	Service Windows (NSSM) en production, fenêtre console en développement

Worker CPU (Celery)	celery -A app.tasks worker -Q cpu -P solo (ou gevent)	Service Windows (NSSM)
Worker GPU (Celery)	celery -A app.tasks worker -Q gpu -P solo --concurrency=1	Service Windows (NSSM), un service par GPU disponible
Scheduler (Celery beat)	celery -A app.tasks beat	Service Windows (NSSM)
PostgreSQL	Service Windows natif (installé par l'installateur PostgreSQL)	Service Windows (démarrage automatique)
Memurai (Redis)	Service Windows natif (installé par l'installateur Memurai)	Service Windows (démarrage automatique)
Nginx	nginx.exe (reverse proxy + fichiers statiques du frontend)	Service Windows (NSSM) ou tâche planifiée au démarrage de session

18.4 Scripts de lancement automatisés (.bat / .ps1)

Trois scripts principaux sont fournis à la racine du dépôt, permettant à un studio de démarrer l'application en un double-clic, sans connaissance technique préalable. Le script PowerShell (.ps1) constitue l'implémentation de référence (plus robuste, gestion d'erreurs typée) ; le script .bat associé est un lanceur minimal qui invoque le .ps1 avec les droits d'exécution appropriés, pour les utilisateurs qui préfèrent un double-clic direct sans configurer la politique d'exécution PowerShell manuellement.

Script	Rôle détaillé
install.ps1 / install.bat	Installation initiale : vérifie/installe les prérequis (§18.2) via winget lorsque possible, crée l'environnement virtuel Python (venv), installe les dépendances (requirements.txt, avec détection GPU pour installer la variante PyTorch/CUDA adéquate), installe les dépendances frontend (npm ci) et lance le build de production des assets statiques, initialise la base de données (création + migrations Alembic), crée un fichier .env local à partir d'un .env.example si absent.
start.ps1 / start.bat	Démarrage de l'application : vérifie que PostgreSQL et Memurai (Redis) sont démarrés (démarré les services Windows si nécessaire via Start-Service), active le venv Python, lance l'API FastAPI (Uvicorn), lance le(s) worker(s) Celery (CPU et GPU si disponible), lance Nginx pour servir le frontend, puis ouvre automatiquement le navigateur par défaut sur http://localhost:8080 . Chaque processus est lancé dans sa propre fenêtre console nommée pour un diagnostic visuel immédiat en environnement de développement.
stop.ps1 / stop.bat	Arrêt propre de l'ensemble des processus applicatifs démarrés par start.ps1 (arrêt gracieux des workers Celery en cours de tâche avant

	coupure, fermeture de l'API, arrêt de Nginx) sans arrêter PostgreSQL/Memurai qui restent disponibles comme services système.
install-service.ps1 (optionnel, production)	Enregistre l'API, les workers et Nginx comme services Windows persistants via NSSM, avec redémarrage automatique en cas de plantage et démarrage automatique au boot du serveur — recommandé pour un déploiement serveur non supervisé.

Extrait de logique du script start.ps1 (pseudocode de référence)

Le tableau ci-dessous décrit la séquence exacte exécutée par start.ps1, à formaliser en script PowerShell réel lors du développement du socle technique (sprint 0, voir §23.1) :

25. Vérifier la présence de Python, Node.js, PostgreSQL et Memurai dans le PATH ou les emplacements d'installation standards ; en cas d'absence, afficher un message d'erreur explicite renvoyant vers install.ps1
26. Démarrer les services Windows PostgreSQL et Memurai s'ils ne sont pas déjà en cours d'exécution (Get-Service / Start-Service)
27. Activer l'environnement virtuel Python (.venv\Scripts\Activate.ps1)
28. Appliquer les migrations Alembic en attente (alembic upgrade head) avant de démarrer l'API
29. Démarrer l'API FastAPI via Uvicorn dans un nouveau processus (Start-Process) avec redirection des logs vers un fichier horodaté dans logs/
30. Détecter la présence d'un GPU NVIDIA (nvidia-smi) et démarrer le worker Celery GPU si disponible, sinon démarrer uniquement le worker CPU
31. Démarrer Nginx avec la configuration prête à l'emploi fournie dans deploy/nginx/nginx.conf
32. Attendre que l'endpoint /health de l'API réponde 200 OK (polling, timeout 30 s) avant d'ouvrir automatiquement le navigateur
33. Afficher un résumé console des services démarrés, de leurs ports et de l'emplacement des fichiers de logs

18.5 Environnements

Environnement	Objectif	Caractéristiques
---------------	----------	------------------

Développement (poste Windows ou Linux)	Développement individuel	install.ps1 puis start.ps1 en local sous Windows, ou équivalents shell (install.sh/start.sh) sous Linux/macOS pour le confort de développement, base PostgreSQL locale, modèles IA en version allégée (Whisper small) pour itérer rapidement, rechargement à chaud de l'API (--reload) ; la cible de livraison reste Windows quel que soit l'OS de développement
Recette / Staging	Validation avant mise en production	Serveur Windows dédié (ou VM), configuration miroir de la production à échelle réduite, données anonymisées, services enregistrés via install-service.ps1
Production	Service aux utilisateurs finaux	Serveur(s) Windows Server, services Windows persistants (NSSM), sauvegardes automatisées planifiées (Planificateur de tâches Windows), monitoring complet

Contrainte de déploiement — environnement de développement multi-OS (mise à jour du cadrage) Si le développement est réalisé sur une machine Linux (ou macOS), l'équipe peut utiliser des outils compatibles avec cet environnement pour son confort de travail quotidien (terminal, gestionnaire de paquets, scripts shell équivalents à install.ps1/start.ps1, etc.). Cette latitude ne concerne que le poste de développement individuel et ne modifie en rien la cible de livraison : le logiciel final — backend, workers, scripts d'installation et de lancement — doit rester utilisable nativement sous Windows, conformément au chapitre 18 et sans dépendance à Docker ni à toute forme de conteneurisation. Toute bibliothèque ou outil introduit pendant le développement doit donc disposer d'un équivalent Windows natif documenté, et les tests de recette avant livraison doivent systématiquement être exécutés sur un environnement Windows cible afin de valider la portabilité réelle du logiciel.

18.6 Scalabilité en environnement Windows natif

- Montée en charge verticale prioritaire (serveur Windows dimensionné avec plusieurs GPU) plutôt qu'un autoscaling horizontal dynamique de type Kubernetes, non pertinent hors conteneurisation
- Possibilité de répartir les workers GPU sur plusieurs serveurs Windows

physiques/VM distincts, chacun pointant vers la même base PostgreSQL et le même broker Memurai centralisés, avec un répartiteur de charge (Nginx ou solution réseau dédiée) devant plusieurs instances API • Pour les studios à fort volume, un second serveur Windows dédié uniquement aux workers GPU peut être ajouté sans modification du code applicatif, simplement en pointant sa configuration vers l'API/la base centrales

18.7 Sauvegarde et reprise d'activité (PRA/PCA)

- Sauvegardes automatiques quotidiennes de PostgreSQL via pg_dump planifié (Planificateur de tâches Windows), rétention 30 jours, test de restauration mensuel
- Copie planifiée du dossier de stockage des médias/exports vers un emplacement distant (partage réseau, stockage objet cloud) pour se prémunir d'une panne matérielle locale
- Objectifs cibles : RPO (perte de données maximale tolérée) ≤ 1 heure, RTO (temps de reprise) ≤ 4 heures
- Procédure de reprise documentée (réinstallation via install.ps1 sur une machine de secours + restauration des sauvegardes) et testée au moins une fois par semestre

19. CI/CD

19.1 Outillage

GitHub Actions (ou GitLab CI selon l'hébergement du code retenu) orchestre les pipelines d'intégration et de déploiement continus, avec une stratégie de branches basée sur trunk-based development assoupli (branches de fonctionnalité courtes, revue de code obligatoire, merge sur main protégé). Les jobs d'intégration s'exécutent sur des runners windows-latest afin de valider en continu la compatibilité Windows native de l'application, conformément à la contrainte de déploiement sans conteneurisation.

19.2 Pipeline d'intégration continue

34. Lint & format (ruff/black pour le backend Python, ESLint/Prettier pour le frontend JS)

35. Analyse statique de sécurité (bandit pour Python, scan de dépendances pip-audit/npm audit)
36. Tests unitaires backend (pytest, couverture minimale imposée ex. 80 % sur les modules domain/services) exécutés sur runner Windows
37. Tests unitaires frontend (Web Test Runner ou Vitest pour les modules JS natifs)
38. Tests d'intégration API (contre une instance PostgreSQL/Memurai éphémère installée sur le runner de test, sans conteneur)
39. Exécution du script install.ps1 en mode silencieux sur runner windows-latest afin de valider la procédure d'installation à chaque changement (test de non-régression du script lui-même)
40. Packaging de l'artefact de livraison (archive .zip versionnée contenant le code backend, les assets frontend buildés, les scripts .bat/.ps1 et les fichiers de configuration par défaut)
41. Tests end-to-end critiques (Playwright) sur les parcours clés : import → pipeline → édition → export

19.3 Pipeline de déploiement continu

- Déploiement automatique en environnement de recette à chaque merge sur main : copie de l'archive de livraison sur le serveur Windows de recette, exécution de install.ps1 -Update puis redémarrage des services via NSSM
- Déploiement en production déclenché manuellement (approbation) ou automatiquement selon une politique de release définie par l'équipe ; les services étant enregistrés via NSSM, la bascule se fait par arrêt/mise à jour des binaires/redémarrage séquencé plutôt que par bascule d'image (stratégie adaptée à l'absence de conteneurisation)
- Migrations de base de données exécutées automatiquement en amont du redémarrage applicatif (alembic upgrade head, avec verrouillage pour éviter les exécutions concurrentes)
- Conservation de l'archive de la version précédente sur le serveur pour permettre un rollback rapide (restauration des binaires + alembic downgrade si nécessaire) en cas d'échec des vérifications post-déploiement
- Feature flags pour

l'activation progressive des fonctionnalités sensibles (ex. synchronisation labiale V1.5) sans nécessiter de nouveau déploiement

19.4 Qualité continue

- Suivi de la dette technique et de la couverture de tests dans le temps (SonarQube ou équivalent)
- Revue de code obligatoire (2 approbateurs minimum sur les modules critiques : sécurité, pipeline IA, facturation)
- Changelog généré automatiquement à partir des commits conventionnels (Conventional Commits) pour la traçabilité des releases

20. Roadmap

20.1 Découpage en phases

Phase	Horizon	Contenu
MVP	M0 → M4	Import vidéo, extraction audio, transcription + diarisation + alignement, génération automatique basique de bande rythmo, éditeur fonctionnel, export SRT/VTX/PDF, gestion utilisateurs, studios/projets, authentification
V1	M4 → M7	Détection respirations/pauses, détection émotions/intentions, profils typographiques personnalisables, exports professionnels étendus (EBU-STL, format Cavena), collaboration multi-utilisateurs, commentaires
V1.5	M7 → M10	Synchronisation labiale (détection faciale), tableau de bord studio avancé, recherche full-text, amélioration continue des modèles (feedback loop)
V2	M10 → M15	SSO Enterprise, édition collaborative temps réel avancée, séparation de sources audio (dialogue/musique/effets), API publique pour intégrations tierces, application mobile de consultation
V3+	M15+	Assistance à la traduction/adaptation, marketplace de profils typographiques par pays, IA de suggestion de découpage de répliques avancée, module de doublage assisté par IA (exploratoire)

20.2 Jalons clés (Milestones)

- **M0** — Cadrage validé, environnements d'infrastructure provisionnés, équipe constituée.
- **M2** — Pipeline IA bout-en-bout fonctionnel en environnement de développement (transcription → diarisation → alignement).
- **M4** — MVP déployé en environnement de recette, premiers tests avec un studio partenaire pilote.
- **M5** — Lancement commercial bêta fermée (3 à 5 studios pilotes).
- **M7** — V1 disponible, ouverture commerciale élargie.
- **M10** — V1.5 avec synchronisation labiale, objectif de 20 studios actifs.

- **M15** — V2, objectif de 50 studios actifs (cf. §1.3).

21. Backlog Complet et User Stories

Le backlog est organisé en Epics, déclinées en User Stories au format « En tant que [rôle], je veux [besoin], afin de [bénéfice] », priorisées selon la méthode MoSCoW (Must have *Should have* Could have *Won't have for now*). *Cette liste constitue la base d'initialisation du backlog produit (JiraLinear) et n'est pas exhaustive — elle sera enrichie en continu.*

21.1 Epic — Import et traitement média

ID	User Story	Priorité	Estimation
US-001	En tant qu'utilisateur, je veux importer une vidéo par glisser-déposer afin de démarrer rapidement un projet	Must	3 pts
US-002	En tant qu'utilisateur, je veux voir la progression de mon upload afin de savoir combien de temps attendre	Must	2 pts
US-003	En tant que système, je dois extraire automatiquement la piste audio de la vidéo importée	Must	3 pts
US-004	En tant qu'utilisateur, je veux être averti si mon fichier est dans un format non supporté afin de corriger rapidement	Must	2 pts
US-005	En tant qu'utilisateur, je veux importer plusieurs pistes audio (VO, versions alternatives) afin de comparer les synchronisations	Should	5 pts
US-006	En tant qu'utilisateur, je veux reprendre un upload interrompu afin de ne pas recommencer depuis le début	Should	5 pts

21.2 Epic — Transcription et analyse IA

ID	User Story	Priorité	Estimation
US-010	En tant qu'utilisateur, je veux obtenir une transcription automatique horodatée au mot près afin de gagner du temps sur le repérage	Must	8 pts
US-011	En tant qu'utilisateur, je veux voir les différents locuteurs identifiés automatiquement afin de ne pas	Must	8 pts

	les distinguer manuellement		
US-012	En tant qu'utilisateur, je veux pouvoir renommer et fusionner des locuteurs mal détectés afin de corriger les erreurs de diarisation	Must	5 pts
US-013	En tant qu'utilisateur, je veux voir les respirations détectées automatiquement afin de les utiliser comme points d'appui	Should	8 pts
US-014	En tant qu'adaptateur, je veux voir un indicateur de débit d'élocution par réplique afin d'ajuster la longueur du texte	Should	5 pts
US-015	En tant que directeur artistique, je veux voir les émotions détectées par réplique afin d'anticiper la direction d'acteur	Could	8 pts
US-016	En tant qu'utilisateur, je veux voir un score de confiance par réplique afin de prioriser ma relecture	Must	3 pts

21.3 Epic — Génération et édition de la bande rythmo

ID	User Story	Priorité	Estimation
US-020	En tant qu'utilisateur, je veux générer automatiquement une première version de la bande rythmo afin de gagner un temps considérable	Must	13 pts
US-021	En tant que calligraphe, je veux déplacer et redimensionner une réplique dans la timeline afin d'ajuster précisément le calage	Must	8 pts
US-022	En tant que calligraphe, je veux scinder ou fusionner des répliques afin de correspondre au découpage souhaité	Must	5 pts
US-023	En tant que calligraphe, je veux appliquer des codes typographiques (italique, majuscules, crochets) afin de respecter les conventions du studio	Must	5 pts
US-024	En tant qu'utilisateur, je veux annuler/rétablir mes actions afin de corriger sereinement mes erreurs	Must	5 pts
US-025	En tant qu'utilisateur, je veux que mes modifications soient sauvegardées automatiquement afin de ne jamais perdre mon travail	Must	5 pts
US-026	En tant que chef de projet, je veux consulter l'historique des versions de la bande rythmo afin de	Should	8 pts

	revenir en arrière si nécessaire		
US-027	En tant qu'utilisateur, je veux personnaliser mes raccourcis clavier afin d'adapter l'outil à mes habitudes	Could	3 pts

21.4 Epic — Collaboration

ID	User Story	Priorité	Estimation
US-030	En tant qu'utilisateur, je veux inviter un collègue sur un projet afin de collaborer	Must	3 pts
US-031	En tant qu'utilisateur, je veux commenter une réplique spécifique afin d'échanger sans quitter l'éditeur	Should	5 pts
US-032	En tant que directeur artistique, je veux valider formellement une bande rythmo afin de la verrouiller pour l'enregistrement	Must	5 pts
US-033	En tant qu'utilisateur, je veux voir qui édite quelle réplique en temps réel afin d'éviter les conflits	Should	8 pts

21.5 Epic — Export

ID	User Story	Priorité	Estimation
US-040	En tant qu'utilisateur, je veux exporter la bande rythmo en PDF calligraphié afin de la fournir au studio d'enregistrement	Must	8 pts
US-041	En tant qu'utilisateur, je veux exporter au format EBU-STL/Cavena afin de rester compatible avec mes outils historiques	Must	8 pts
US-042	En tant qu'utilisateur, je veux exporter en SRT/VTT afin de réutiliser les données pour du sous-titrage	Should	3 pts
US-043	En tant qu'utilisateur, je veux exporter un rapport d'analyse IA (qualité, confiance) afin de documenter le contrôle qualité	Could	5 pts

21.6 Epic — Administration, sécurité, facturation

ID	User Story	Priorité	Estimation
US-050	En tant qu'administrateur studio, je veux gérer les rôles de mes collaborateurs afin de contrôler les accès	Must	5 pts

US-051	En tant qu'administrateur studio, je veux configurer un profil typographique personnalisé afin de respecter nos conventions internes	Must	8 pts
US-052	En tant qu'utilisateur, je veux activer la double authentification afin de sécuriser mon compte	Should	5 pts
US-053	En tant qu'administrateur studio, je veux consulter ma consommation (minutes traitées, stockage) afin de suivre mon abonnement	Should	5 pts
US-054	En tant qu'administrateur studio, je veux configurer le SSO Enterprise afin de simplifier l'accès à mes équipes	Could	13 pts

21.7 Synthèse de vélocité prévisionnelle

En considérant une équipe de développement type (voir §22 Estimation budgétaire) livrant en moyenne 30 à 40 points de complexité par sprint de deux semaines, le périmètre MVP (Epics 21.1 à 21.3, sous-ensemble Must have, soit environ 90 points) est réalisable en 6 à 8 sprints, cohérent avec l'horizon M4 fixé en roadmap.

22. Estimation Budgétaire

22.1 Hypothèses

L'estimation ci-dessous couvre la période de conception et développement jusqu'au lancement de la V1 (M0 à M7, soit environ 7 mois), en France, avec une équipe mixte interne/freelance de taille adaptée à un projet SaaS B2B de complexité IA élevée. Les montants sont indicatifs (coûts chargés moyens constatés sur le marché français, à ajuster selon la localisation réelle de l'équipe et le mode de contractualisation).

22.2 Composition d'équipe recommandée

Rôle	ETP	Coût mensuel chargé indicatif	Période d'intervention
Product Owner / Chef de projet	1	7 000 – 9 000 €	M0 → M7 (continu)
Lead développeur backend (Python/FastAPI)	1	8 000 – 10 000 €	M0 → M7 (continu)
Développeur backend	1	6 500 – 8 500 €	M1 → M7
Ingénieur ML / MLOps (pipeline IA)	1,5	8 500 – 11 000 €	M0 → M7 (continu)
Développeur frontend (JS natif)	1,5	6 500 – 8 500 €	M0 → M7 (continu)
Designer UX/UI	0,5	6 000 – 7 500 €	M0 → M4, ponctuel ensuite
Ingénieur DevOps / Infra	0,5	7 500 – 9 500 €	M0 → M7 (croissant)
QA / Testeur	0,5	5 000 – 6 500 €	M2 → M7
Expert métier doublage (consultant)	0,2	600 – 900 € / jour	Ponctuel, ateliers et recette

22.3 Estimation des coûts de développement (M0 à M7)

Poste	Fourchette basse	Fourchette haute
Masse salariale / prestations équipe (7 mois)	310 000 €	420 000 €

Conception UX/UI et tests utilisateurs	20 000 €	35 000 €
Infrastructure cloud (dev + recette + GPU d'entraînement/test)	15 000 €	30 000 €
Licences et services tiers (transcription cloud de secours, monitoring APM, outillage CI)	8 000 €	18 000 €
Audit de sécurité / pentest pré-lancement	8 000 €	15 000 €
Marge de contingence (15 %)	54 000 €	78 000 €
TOTAL indicatif MVP → V1	≈ 415 000 €	≈ 596 000 €

22.4 Coûts d'exploitation récurrents (post-lancement, mensuel, à 20 studios actifs)

Poste	Fourchette basse	Fourchette haute
Infrastructure cloud production (API, DB, stockage)	1 500 €	3 500 €
Calcul GPU (traitement IA à l'usage)	3 000 €	9 000 €
Monitoring, sécurité, sauvegardes	800 €	1 500 €
Support et maintenance (part d'ETP dédiée)	6 000 €	10 000 €
TOTAL mensuel indicatif	≈ 11 300 €	≈ 24 000 €

Note méthodologique

Ces montants sont des ordres de grandeur destinés au cadrage budgétaire initial ; ils doivent être affinés lors du chiffrage détaillé par l'équipe retenue, en particulier le coût GPU qui dépend fortement du volume réel traité et du choix self-hosted vs cloud à la demande.

23. Planning

23.1 Planning macro (Gantt textuel)

Phase	Durée	Contenu
Cadrage & conception	4 semaines (M0)	Validation du cahier des charges, maquettes UX, choix définitifs d'infrastructure, mise en place des environnements

Socle technique	4 semaines (M1)	Squelette backend FastAPI, authentification, modèle de données initial, squelette frontend, CI/CD de base
Pipeline IA v1	6 semaines (M1-M2)	Intégration Whisper/WhisperX/pyannote, orchestration Celery, premiers résultats bout-en-bout
Éditeur bande rythmo	6 semaines (M2-M3)	Timeline, waveform, édition des répliques, moteur de génération automatique
Export & finitions MVP	3 semaines (M3-M4)	Exports SRT/VTT/PDF, gestion projets/utilisateurs/studios, durcissement sécurité
Recette & bêta pilote	3 semaines (M4)	Tests avec studio(s) partenaire(s), corrections, ajustements UX
V1 — respirations, émotions, profils typo	8 semaines (M5-M6)	Modules IA avancés, personnalisation typographique, collaboration
V1 — exports professionnels étendus	4 semaines (M6-M7)	Formats EBU-STL, Cavena, rapport qualité
Lancement commercial V1	2 semaines (M7)	Communication, onboarding des premiers studios payants

23.2 Rituels de gestion de projet

- Sprints de 2 semaines (méthodologie Scrum adaptée), revue de sprint avec démonstration systématique
- Comité de pilotage mensuel avec les parties prenantes métier (studio(s) pilote(s))
- Rétrospective d'équipe à chaque fin de sprint
- Point hebdomadaire dédié à l'avancement du pipeline IA (risques techniques spécifiques, qualité des modèles)

24. Analyse des Risques

24.1 Matrice des risques

Risque	Probabilité	Impact	Mesures de mitigation
Précision insuffisante de la transcription/alignement sur accents ou voix bruitées	Moyenne	Élevé	Fallback multi-modèles (WhisperX/MFA), amélioration continue, mise en avant claire du score de confiance, édition manuelle toujours possible sans friction
Coût GPU supérieur aux prévisions à grande échelle	Moyenne	Élevé	Quantification des modèles, mutualisation/scheduling intelligent, tarification à l'usage répercutée si nécessaire, exploration de modèles plus légers
Résistance au changement des calligraphes rythmo (peur de substitution)	Élevée	Moyen	Positionnement clair « assistant, pas remplaçant », implication des professionnels dès la conception, communication sur le gain de temps plutôt que la suppression de poste
Fuite de contenu sous embargo (incident de sécurité)	Faible	Très élevé	Chiffrement bout-en-bout, watermarking, audit de sécurité régulier, politique de rétention stricte, assurance cyber
Non-conformité aux formats propriétaires historiques (Cavena etc.)	Moyenne	Élevé	Travail de rétro-ingénierie des formats en amont avec studios partenaires, tests de compatibilité systématiques
Dépendance à des fournisseurs IA tiers (API cloud) en cas d'indisponibilité	Faible	Moyen	Priorité aux modèles self-hosted (Whisper, pyannote), circuit breaker et fallback local (§6.4)
Retard de développement du pipeline IA (complexité sous-estimée)	Moyenne	Moyen	Time-boxing avec points d'étape hebdomadaires dédiés, priorisation MoSCoW stricte, MVP volontairement réduit sur les modules IA les plus expérimentaux (émotion, synchro labiale)
Difficulté de recrutement de profils ML/MLOps spécialisés audio	Moyenne	Moyen	Anticipation du recrutement dès le cadrage, recours possible à des freelances spécialisés, formation interne

Adoption commerciale plus lente que prévu	Moyenne	Élevé	Programme bêta avec studios pilotes dès le MVP, tarification flexible d'entrée, ROI démontrable chiffré (temps gagné)
Évolution réglementaire (IA Act européen, RGPD)	Moyenne	Moyen	Veille juridique continue, documentation de conformité IA Act (classification du système, transparence sur l'usage de l'IA), DPIA anticipée

24.2 Risques techniques spécifiques au domaine audiovisuel

- **Variabilité de la qualité audio source** — mixages professionnels avec musique/effets forts pouvant dégrader la transcription : mitigation par séparation de sources en V2 et réglages de sensibilité configurables.
- **Diversité linguistique et dialectale** — le doublage français traite aussi des versions belges, québécoises, suisses avec variations phonétiques : tests de robustesse multi-accent à intégrer dès le plan de recette.
- **Contenus musicaux (comédies musicales, chansons)** — cas particulier nécessitant un traitement dédié, hors périmètre précis du MVP mais à cadrer explicitement avec les studios pilotes pour éviter les attentes déçues.

24.3 Gouvernance des risques

Un registre des risques est tenu à jour et revu à chaque comité de pilotage mensuel (§23.2), avec un propriétaire de risque désigné pour chaque item à probabilité ou impact élevé, et un plan d'action daté associé.

25. Évolutions Futures

25.1 Assistance à l'adaptation et à la traduction

Intégration d'une assistance IA à l'adaptation (proposition de reformulations respectant le calibrage syllabique disponible, à partir d'un script source traduit littéralement), en complément et non en remplacement du travail de l'adaptateur — fonctionnalité à forte valeur perçue mais nécessitant une validation qualité approfondie avant mise à disposition.

25.2 Extension à d'autres marchés linguistiques

La bande rythmo est utilisée principalement en France et dans plusieurs pays francophones et européens (procédés dérivés). Une extension du support multilingue de l'interface et des profils typographiques par pays (conventions italiennes, espagnoles, allemandes de doublage) est envisageable à partir de la V2/V3, en s'appuyant sur l'architecture déjà multilingue du pipeline de transcription.

25.3 Marketplace de profils et de ressources

Mise en place d'une bibliothèque communautaire de profils typographiques et de règles de segmentation partagées entre studios (opt-in), accélérant l'onboarding de nouveaux clients.

25.4 API publique et écosystème d'intégrations

Ouverture d'une API publique documentée permettant l'intégration de RythmoAI dans les outils de gestion de production existants des grands studios et diffuseurs (ERP de production, plateformes de gestion de droits), ainsi que des webhooks pour automatiser les workflows (notification de fin de traitement, déclenchement d'export automatique).

25.5 Approfondissement de l'IA

- Réentraînement/fine-tuning de modèles propriétaires sur corpus doublage FR (avec consentement contractuel des studios) pour dépasser la précision des modèles génériques
- Amélioration de la détection de synchronisation labiale (modèles dédiés entraînés spécifiquement sur des rushs de doublage)
- Suggestions de découpage de répliques encore plus fines tenant compte du jeu d'acteur (rythme théâtral, respiration dramatique) au-delà des règles heuristiques actuelles

25.6 Application mobile compagnon

Une application mobile (consultation, validation, commentaires en déplacement) pourra être envisagée en V2+ pour les rôles de supervision

(directeurs artistiques, chefs de projet), sans reproduire les fonctions d'édition fine réservées au poste de travail desktop.

A. Annexes

A.1 Glossaire

Terme	Définition
Bande rythmo / LRD	Bande de texte défilante synchronisée à l'image utilisée par le comédien de doublage pendant l'enregistrement
Repérage	Découpage préalable de la vidéo en boucles/répliques exploitables pour la calligraphie
Calligraphie rythmo	Métier consistant à tracer manuellement la bande rythmo en respectant le calage labial et les codes du doublage
Diarisation	Processus d'identification et de séparation des différents locuteurs dans un flux audio
Forced alignment (alignement forcé)	Technique associant précisément chaque mot d'une transcription à ses timestamps exacts dans le signal audio
WER (Word Error Rate)	Taux d'erreur de mots, métrique standard d'évaluation de la qualité d'une transcription automatique
EBU-STL	Format de sous-titrage standardisé par l'Union Européenne de Radio-Télévision, référence dans l'audiovisuel professionnel
VAD (Voice Activity Detection)	Détection automatique des zones de parole vs silence dans un signal audio

A.2 Détail des formats d'export professionnels

Format	Usage principal	Contenu exporté
PDF calligraphié	Impression / lecture en cabine d'enregistrement	Bande rythmo mise en page avec codes typographiques, timecodes de référence
EBU-STL (étendu)	Interopérabilité broadcast standard	Répliques horodatées, styles, métadonnées locuteur
Format Cavena / .rythmo	Compatibilité avec logiciels historiques de calligraphie	Structure propriétaire reconstituée (repères, textes, timings)
SRT / VTT (étendus avec métadonnées)	Réutilisation pour sous-titrage ou prévisualisation web	Texte horodaté, locuteur en commentaire, styles basiques
JSON structuré (API)	Intégration avec des outils tiers ou scripts internes studio	Export complet et fidèle de toutes les données du projet (mots, répliques, métadonnées IA)

XML propriétaire RythmoAI	Sauvegarde/restauration complète d'un projet entre environnements	Snapshot intégral incluant historique de versions
---------------------------	---	---

A.3 Références et sources

Les choix technologiques de ce document s'appuient sur l'état de l'art public des solutions de traitement de la parole et de la vidéo à la date de rédaction (modèles Whisper d'OpenAI, WhisperX, pyannote.audio, écosystème FastAPI/PostgreSQL/Celery), ainsi que sur une analyse du fonctionnement du métier de calligraphe rythmo et du processus de doublage tel que pratiqué dans l'industrie francophone.

A.4 Prochaines étapes immédiates

42. Validation formelle de ce cahier des charges par les parties prenantes (direction, studio(s) pilote(s))
43. Constitution de l'équipe projet (voir §22.2)
44. Atelier de cadrage technique détaillé (choix définitif d'hébergement, environnements)
45. Recrutement/identification d'un studio partenaire pilote pour les phases de recette
46. Lancement du sprint 0 (mise en place des environnements et du socle technique)